

Routing Language Models For Various Medical Scenario Benchmarks

Bence Danko

dept. of Applied Data Science

San Jose State University

San Jose, USA

bence.danko@sjsu.edu

Abstract—Large language model (LLM) agents are expected to operate and adapt to a wide variety of scenarios and environments. In the medical domain, benchmarks have emerged to test zero-shot question answering, multi-turn conversation guard-railing, and tool-use in production environments, each requiring their own suite for evaluation and scoring. However, many existing benchmarks have either saturated due to the rapid advancement of frontier models or suffered from data contamination during pretraining. Consequently, developing systems that can dynamically route these predictable tasks to smaller, specialized models represents a valuable research area. In this work, we unify three evaluation frameworks in order to evaluate dynamic routing. We demonstrate that by using a trained DistilBERT model, queries can be routed to appropriate scenario handlers to reduce inference costs. We also show that benchmark unification is a robust way to evaluate routing systems for scenario variability, and that we do so under an upcoming common standard for future reproducibility.

Index Terms—dynamic routing, model cascading, model routing, DistilBERT

I. INTRODUCTION

In recent years, several major NLP and LLM-focused benchmarks have emerged to test language models on knowledge and proficiency in the medical domain. Early medical benchmarks and datasets were based on zero-shot question-answering (QA). However, for practical medical production usage, new evaluation frameworks have emerged that test not just factuality, but strict human-aligned guard-railing, and real-world medical systems interaction (automated tool use).

A. Question-Answering Benchmarks

Medical multiple-choice question-answering (QA) was an early metric for evaluating parametric clinical knowledge and reasoning capabilities of language models. Early datasets primarily focused on retrieving facts from biomedical literature or answering multiple-choice questions derived from medical board examinations. For instance, PubMedQA [1], released in 2019, challenged models to reason over biomedical research texts to provide definitive answers. MedQA [2] was introduced in 2020 to evaluate models on complex USMLE-style questions, requiring deeper clinical reasoning. The medical subset of the MMLU benchmark introduced in 2021 [3] further tested across professional and academic medical subjects. By 2022, MedMCQA [4] provided a large multi-subject dataset derived from Indian medical entrance exams.

These QA benchmarks capably test for encyclopedic knowledge and binary accuracy, but are saturating due to the rapid advancement of frontier models. Additionally, traditional QA does not capture the complexities of deploying LLMs in live clinical environments.

B. Behavior Guardrail and HealthBench

Medical scenario evaluations have expanded beyond QA correctness. Benchmarks such as HealthBench from OpenAI [5] evaluates behavior guardrail in multi-turn dialogues. MedAgentBench [6] tests real-world medical systems interaction and automated tool use.

Assessing conversational alignment and tool-use is highly subjective. It requires robust LLM-as-a-Judge scoring systems to effectively evaluate the nuances of patient interaction, safety constraints, and proper Fast Healthcare Interoperability Resources (FHIR) database state manipulation.

C. Common Unified Benchmark Environments

Despite the growing breadth of evaluation frameworks, the community has not yet adopted a single unified technical approach to conducting reproducible evaluations. There is a practical research need for such a unified, multi-scenario benchmark. Frameworks like CUBE [7] represent a newly proposed library standard designed to abstract otherwise incompatible benchmarks across disparate runtime environments. Though not yet widely adopted, this paper implements all evaluations under CUBE wrappers in an attempt to contribute to open-source benchmark standardization efforts.

D. Routing LLMs

In practical applied LLM systems, for economic needs, router-based LLM orchestrations that route requests based on query complexity and needs. Simple queries route to less powerful but capable models, whereas more complex requests need to be allocated to more powerful agentic harnesses and tool-enabled prompts.

II. RELATED WORKS

RouteLLM is one formalization of routing, where researchers optimize trade-offs between cost without sacrificing response quality [8]. In their framework, the researchers explore out-of-domain generalization capabilities and found

that routing models are indeed adaptable to out-of-domain distribution shifts. Another form of routing frameworks is cascading routing systems. These sequentially invoke progressively larger models until a specific confidence thresholds or criteria are met. However, Hybrid LLM has previously demonstrated that predictive, single-shot upstream proves cheaper and more deterministic than complex or iterative cascading routing architectures [9]. This single-shot gating mechanism is our inspiration for our simple specialized classification approach.

III. METHODOLOGY

A. Question Answering Evaluations

We initially evaluated frontier model performance on various QA benchmarks to measure disparities in performance. Models were prompted to provide both a multiple-choice answer and confidence in structured JSON.

The mathematical formulation for evaluating model confidence alongside correctness uses Expected Calibration Error (*ECE*). Predictions are partitioned into M equally spaced confidence intervals (or bins) spanning across $[0, 1]$. It is top-label confidence calibration. The model predicts one answer, it reports one confidence. We check answer correctness, then measure among all example confidence bins, measuring if it was correct an equivalent frequency.

$$ECE = \sum_{m=1}^M \frac{|B_m|}{n} |\text{acc}(B_m) - \text{conf}(B_m)|$$

A minimal *ECE* value signals an aligned model, preventing over-confident hallucinations. By using this metric, we measure how well-calibrated the models generated, self-reported text confidences are.

B. Question Answering RAG Implementation

Following initial QA evaluations, we followed up with a RAG implementation, using pre-chunked PubMed abstracts and foundational textbooks from an existing implementation from MedRAG [10], using FAISS (Facebook AI Similarity Search) [11] for vector similarity searches.

C. Unifying Medical Benchmarks and Routing System

This is our proposed novel contribution. We split and evaluate on subsets of MMLU-Medical, HealthBench, and MedAgentBench, the details of which can be found in Table I. We chose these sets in particular because we believe them to be relatively out of domain from each other, in which MMLU-Medical was found to be sufficiently easy for frontier models to answer, where HealthBench proved difficult and introduces multi-turn conversation. MedAgentBench introduces MCP tool-use requirements, and requires a separately running MCP server implementation accessible via network calls. This was exposed via a FastAPI handler and API.

Our system is summarized under Figure 1. A query passes through a DistilBERT router. The router predicts a preferred model selection that was prepared: MMLU-Medical

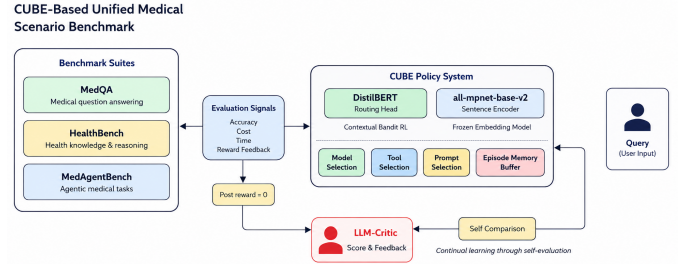


Fig. 1. A diagram of our proposed approach. A query is passed through a DistilBERT router that classifies to predict preferred model selection, tool selection, and prompt selection. An additional episodic memory mechanism, to retrieve zero reward (failed) memory traces is also presented.

queries routed to gpt-oss-120b, HealthBench to gemini-3-flash-preview, and MedAgentBench to gemma-4-31b-it. The model also classifies between tool use (binary: none, or fhir-enabled) and scenario recognition for prompt selection between the 3 benchmark scenarios. In our final evaluations, only the optimal model routing decision impacted on model execution, allowing the system to choose for preferred, cost effective models.

TABLE I
DATASET SPLIT, INPUTS, OUTPUTS, AND REWARD SCORING ACROSS EVALUATION BENCHMARKS

Data Split	Total	MMLU-Med	HealthBench	MedAgentBench
Training	1,350	800	400	150
Validation	450	200	200	50
Testing	1,371	871	400	100
Inputs		question, choices_json	messages_json	instruction, context, tools_json
Outputs		QA Answer [A, B, C, D]	Conversation Completion	FHIR database state, Conversation
Reward Scoring		0 or 1 (hard coded check)	LLM Response (0 to 1, rubric grade)	LLM Judge (0 or 1, success or fail)

IV. DATA AND PREPROCESSING

A. Episodic RAG

During training run evaluations, failed episodes were recorded as memory candidates. Each text contains the benchmark, task id, reward, failure mode, generic benchmark-specific advice, original task request, model prediction summary, judge/scorer feedback and transcript excerpts. Text is chunked by recursive character splitting, using a chunk boundary of 900 characters with overlap of 120 characters. Chunks are embedded using sentence-transformers/all-mpnet-base-v2, and on evaluation, we use top-3 retrieval using cosine similarity.

B. Routing Dataset

The training dataset for the routing model was prepared by mapping model queries to model labels, binary tool enablement, and prompt labels (benchmark scenarios).

Queries were assembled using context from each benchmark:

```
Task:
...

Context:
...

Answer options:
...
```

Listing 1. Prompt assembly

The structure of the prompt admittedly made classification easy from content shape. Multiple-choice questions have answer options, MedAgentBench has chart-like context plus operational instructions, and HealthBench has free-form user conversation and request text.

V. EXPERIMENTAL DESIGN

distilbert-base-uncased was used as the base architecture, in which we attached a basic head to classify our target labels. The training workflow runs across 4 epochs with an optimized mini-batch size of 32. Weights are updated via an AdamW optimizer using a set learning rate of 2×10^{-5} . Table II and Table III document the hyperparameters.

TABLE II
HYPERPARAMETER CONFIGURATION FOR MODEL TRAINING

Parameter	Value
Base Model Architecture	distilbert/distilbert-base-uncased
Total Training Epochs	4
Mini-Batch Size	32
Learning Rate Optimizer	2e-5

TABLE III
CONFIGURATION PARAMETERS FOR TEXT SPLITTING

Parameter	Value
Chunk Size Limit	900 characters
Chunk Overlap Boundary	120 characters
Split Processing Style	Recursive Character Splitting

A. Prompt techniques and LLM-as-a-Judge Queries

```
[
  {
    "role": "system",
    "content": (
      "You are a medical expert. Return ONLY valid JSON in this exact format: "
      "{\\"answer\\":\\"A\\",\\"confidence\\":0.0}."
      "The answer must be A, B, C, or D."
    ),
  },
  {"role": "user", "content": body},
]
```

Listing 2. System and User zero-shot QA prompt

```
Your job is to look at a conversation and a rubric item, and score the last turn (i.e., the last assistant response, or the completion) on compliance.
Return a json object with fields: "explanation" and "criteria_met".
[... standard structural guidelines and examples embedded here ...]
```

Listing 3. Judge Prompt for HealthBench Item Rubric Grading

```
You are an expert in using FHIR functions to assist medical professionals. Based on the question, make one or more tool calls. Format rules:
1. GET url?param_name=param_value1...
2. POST url [payload data in JSON]
3. FINISH([answer1, answer2, ...])
```

Listing 4. MedAgentBench API Prompts

```
[
  {
    "role": "system",
    "content": (
      "You are judging a MedAgentBench task. Compare the model's FINISH payload "
      "against expected outputs. Return JSON with keys: score, success, explanation."
    ),
  },
]
```

Listing 5. MedAgentBench LLM-as-a-Judge

VI. EXPERIMENTAL RESULTS

Table IV outlines our initial QA trials. We were able to compare frontier model capabilities and observationally we measured cost. Notably, though gemini-3-flash outperforms on all tests, all models were comparable had close scores for MMLU-Medical, which inspired us to use it as our QA subset in our unified benchmark, as gpt-oss-120b is measurably faster and cheaper for evaluation.

TABLE IV
MODEL PERFORMANCE EVALUATION ACROSS MEDICAL BENCHMARKS

Evaluation	Model	Accuracy	F1-Macro	ECE
MedQA	gemini-flash-prev	0.9442	0.9438	0.0332
MedQA	gemma-4-31b-it	0.8409	0.8394	0.1344
MedQA	gpt-oss-120b	0.9018	0.9060	0.0632
MedMCQA	gemini-flash-prev	0.8505	0.8499	0.1047
MedMCQA	gemma-4-31b-it	0.7496	0.7475	0.2104
MedMCQA	gpt-oss-120b	0.7352	0.7377	0.1923
PubMedQA	gemini-flash-prev	0.8048	0.6238	0.1443
PubMedQA	gemma-4-31b-it	0.7940	0.5901	0.1709
PubMedQA	gpt-oss-120b	0.6680	0.5800	0.2241
MMLU-Med	gemini-flash-prev	0.9221	0.9198	0.0483
MMLU-Med	gemma-4-31b-it	0.8938	0.8914	0.0868
MMLU-Med	gpt-oss-120b	0.8859	0.8866	0.0819

Table V shows the impact of introducing MedRAG chunks and FAISS, which measurably reduced performance. This is indicative that naive RAG simply adds noise to model QA when imprecise, as we did not carefully optimize it.

Table VI shows our initial test split evaluation performance, and Table VII shows our scores while preparing training

TABLE V
PERFORMANCE COMPARISON OF GPT-OSS-120B WITH AND WITHOUT RAG ACROSS MEDICAL BENCHMARKS

Evaluation Domain	Accuracy	F1-Macro	ECE
MedQA Baseline	0.9018	0.9060	0.0632
MedQA With RAG	0.8995	0.9019	0.0612
MedMCQA Baseline	0.7352	0.7377	0.1923
MedMCQA With RAG	0.7322	0.7333	0.1925
PubMedQA Baseline	0.6680	0.5800	0.2241
PubMedQA With RAG	0.6440	0.5544	0.2413
MMLU-Medical Baseline	0.8859	0.8866	0.0819
MMLU-Medical With RAG	0.8862	0.8868	0.0773

samples for episodic memory data preparation. We get state of the art performance of MMLU-Medical and MedAgent-Bench, while HealthBench struggles. We can notice that the LLM-as-a-Judge system for HealthBench is also significantly expensive, but is the most robust guardrail metric.

TABLE VI
TEST SPLIT BASE PERFORMANCE EVALUATION RESULTS (RUN 1 OF 2)

Model	Benchmark	Mean	Latency	Cost	Tokens
gemini-3-flash	mmlu	0.9265	1.36s	\$0.0831	117,420
	medical				
	healthbench	0.3868	26.15s	\$5.8573	8,394,373
	medagentbench	0.8800	5.80s	\$0.5742	1,058,002

TABLE VII
TRAIN SPLIT EVALUATION RESULTS ACROSS DIFFERENT BENCHMARKS

Model	Benchmark	Mean	Latency	Cost	Tokens
gemini-3-flash	mmlu	0.9200	1.24s	\$0.0771	108,904
	medical				
	healthbench	0.3778	26.06s	\$5.5574	7,927,822
	medagentbench	0.8667	6.24s	\$0.6957	1,660,208

The classification accuracy of our trained routing gate layer across the validation and test datasets is documented in Table VIII.

TABLE VIII
CLASSIFIER PERFORMANCE EVALUATION ACROSS DATA SPLITS

Split	n	Model Intent	Tool Routing	Scenario Match
Train	1,350	99.93%	100.00%	99.85%
Validation	450	100.00%	100.00%	100.00%
Test	1,371	99.93%	100.00%	99.93%

The latency introduced by using the router on a consumer-grade CPU is shown Table IX, comparing routing and memory retrieval components. We can see that either operation is very cheap even on the edge, and contributes to significant cost savings.

TABLE IX
LATENCY METRICS ACROSS DIFFERENT FUNCTIONAL MODES

Mode	Mean	Median	P95	P99	Max
Router only	99 ms	85 ms	196 ms	273 ms	530 ms
Router component	162 ms	158 ms	257 ms	331 ms	392 ms
Memory retrieval	208 ms	180 ms	390 ms	609 ms	844 ms
Combined	370 ms	332 ms	635 ms	959 ms	1174 ms

Finally, Table* X shows all evaluations across both our unrouted, gemini-3-flash-preview baseline, and our routed system. We can see reduced costs in both latency and API costs

VII. DISCUSSION AND LIMITATIONS

As indicated by Table VIII, the fine-tuned intent gate achieved near-perfect classification performance. Though this was likely due to admittedly easy prompting structures, it reveals that even small language models are capable of routing evaluation queries for cost saving and optimization. Basic accuracy was preserved while latency and costs were reduced.

Our episodic memory lookup system was not successful. Our injected prompts caused a slight degradation in accuracy across benchmarks. Additionally, combining routing layers with memory vector lookups raises cumulative system latency to a mean of 370 ms (Max 1174 ms). While this latency remains manageable for offline processes, it may introduce bottlenecks in real-time conversational clinical applications as conversations continue and build up.

A primary limitation of this work relates to evaluation environment unification. Our unification was not a perfect representation of a messy and hard-to-distinguish query system, making our classifier task deceptively easy.

VIII. CONCLUSIONS

This work confirms the feasibility of constructing unified, multi-scenario routing gates across disparate medical evaluation frameworks. We demonstrated that clinical queries can be categorized with high accuracy, minimizing unnecessary inference costs on massive frontier models without significantly degrading accuracy. Our experiments demonstrate that adding context-augmented memory paths does not universally improve task scores and can occasionally distract reasoning models while increasing system latency. Future research will explore creating a more robust unification framework and improvement the unified scenario environment.

REFERENCES

- [1] Q. Jin, B. Dhingra, Z. Liu, W. W. Cohen, and X. Lu, "Pubmedqa: A dataset for biomedical research question answering," 2019. [Online]. Available: <https://arxiv.org/abs/1909.06146>
- [2] D. Jin, E. Pan, N. Oufattole, W.-H. Weng, H. Fang, and P. Szolovits, "What disease does this patient have? a large-scale open domain question answering dataset from medical exams," 2020. [Online]. Available: <https://arxiv.org/abs/2009.13081>
- [3] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, "Measuring massive multitask language understanding," 2021. [Online]. Available: <https://arxiv.org/abs/2009.03300>

TABLE X
TEST SPLIT EVALUATION RESULTS ACROSS DIFFERENT BENCHMARKS

Model Config	Benchmark	Official score mean	Judge mean	Response Latency	Judge Latency	Response Cost	Judge Cost	Response Tokens	Judge Tokens
gemini-3-flash	mmlu_medical	0.9242	—	1.274s	—	\$0.0837	—	117,531	—
	healthbench	0.3907	—	10.231s	17.935s	\$0.8118	\$5.0383	329,681	8,049,743
	medagentbench	0.2900	0.8600	4.075s	1.947s	\$0.4012	\$0.0386	1,020,350	37,337
Routed Network	mmlu_medical	0.8840	—	0.501s	—	\$0.0000	—	348,477	—
	healthbench	0.3789	—	11.367s	23.083s	\$0.7756	\$5.5229	420,002	9,179,169
	medagentbench	0.3548	0.7957	2.902s	1.626s	\$0.3012	\$0.0322	952,973	31,534
Routed + Memory	mmlu_medical	0.8829	—	0.605s	—	\$0.0003	—	750,067	—
	healthbench	0.3615	—	10.735s	22.548s	\$0.7602	\$6.2883	551,179	10,629,937
	medagentbench	0.2000	0.8111	3.053s	1.606s	\$0.3317	\$0.0328	1,039,319	31,972

- [4] A. Pal, L. K. Umapathi, and M. Sankarasubbu, “Medmcqa : A large-scale multi-subject multi-choice dataset for medical domain question answering,” 2022. [Online]. Available: <https://arxiv.org/abs/2203.14371>
- [5] R. K. Arora, J. Wei, R. S. Hicks, P. Bowman, J. Quiñero-Candela, F. Tsimplouras, M. Sharman, M. Shah, A. Vallone, A. Beutel, J. Heidecke, and K. Singhal, “Healthbench: Evaluating large language models towards improved human health,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.08775>
- [6] Y. Jiang, K. C. Black, G. Geng, D. Park, J. Zou, A. Y. Ng, and J. H. Chen, “Medagentbench: A realistic virtual ehr environment to benchmark medical llm agents,” 2025. [Online]. Available: <https://arxiv.org/abs/2501.14654>
- [7] A. Lacoste, N. Gontier, O. Shliazhko, A. Jaiswal, K. Sareen, S. Nanisetty, J. Cabezas, M. D. Verme, O. G. Younis, S. Baratta, M. Avale, I. Kerboua, X. H. Lù, E. Bandel, M. Shmueli-Scheuer, A. Yehudai, L. Choshen, J. Lebensold, S. Hughes, M. Caccia, A. Drouin, S. Reddy, T. Yu, Y. Su, G. Neubig, and D. Song, “Cube: A standard for unifying agent benchmarks,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.15798>
- [8] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica, “Routellm: Learning to route llms with preference data,” 2025. [Online]. Available: <https://arxiv.org/abs/2406.18665>
- [9] D. Ding, A. Mallick, C. Wang, R. Sim, S. Mukherjee, V. Ruhle, L. V. S. Lakshmanan, and A. H. Awadallah, “Hybrid llm: Cost-efficient and quality-aware query routing,” 2024. [Online]. Available: <https://arxiv.org/abs/2404.14618>
- [10] G. Xiong, Q. Jin, Z. Lu, and A. Zhang, “Benchmarking retrieval-augmented generation for medicine,” 2024. [Online]. Available: <https://arxiv.org/abs/2402.13178>
- [11] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, M. Lomeli, L. Hosseini, and H. Jégou, “The faiss library,” 2024.